Test Name	Scenario	Test Steps	Expected Behavior	Actual Behavior	Result
shouldSearchAndOpenProduct	TC1	Search for a product → verify input value, page title, and result count → open the first product → verify title, price, and **Add to Cart** button	Search reflects the entered term; results are returned; product page loads with a valid price and enabled **Add to Cart** button	As expected	Passed
shouldVerifySoldOutAndNotifyMeOnUnavailableVariant	TC1	Navigate directly to a product URL → select an unavailable variant	**Notify Me** option is displayed; **Sold Out** label is visible and disabled	As expected	Passed
shouldHandleSearchWithSpecialCharacters	TC1	Search using a special-character string → verify the input echoes the exact string (`assertEquals`) → expect zero results	Search input matches exactly zero, products are returned	Test failed because the application normalized (lowercased, trimmed, or encoded) the search query and returned products for the special-character search	Failed
shouldHandleEmptySearch	TC1	Submit an empty search → if redirected to the home page, pass; otherwise expect zero search results	User is redirected to the homepage or zero products are displayed	The application redirected to a generic search conditions to fail	Failed
shouldHandleSearchWithLongNumericInput	TC1	Search using a very long numeric string → verify the input	Input value matches the entered text;	As expected	Passed

		value → verify the **No Results** message	**No Results** message is displayed		
shouldHandleSearchWithInvalidInput	TC1	Search using an invalid search term → verify input value → verify **No Results** message	Input value matches the entered text; **No Results** message is displayed	As expected	Passed
shouldVerifyAllSearchResultsContainSearchProduct	TC1	Search a valid keyword → iterate through every returned product title → verify each contains the search term	Every displayed product title contains the searched keyword	One or more products did not contain the search term, because the site also returns related, recommended, or category-based products	Failed
shouldFilterProductsByColor	TC2	Hover **Men** → **Jackets** → record product count → apply a color filter → verify updated count and displayed colors	Product count updates and every displayed product matches the selected color	As expected	Passed
shouldFilterProductsByPrice	TC2	Apply a valid price range filter → verify every displayed product falls within the selected range	All displayed products are within the specified minimum and maximum price	As expected	Passed

shouldNotFilterProductsWithNegativePriceRange	TC2	Enter a negative minimum price with a valid maximum price → verify invalid values are rejected	Negative values are rejected or ignored without applying the filter	The application clamped the negative value to zero, resulting in valid filtered products and causing the `assertFalse` expectation to fail.	Failed
verifyPriceLowToHighSorting	TC3	Open the **Men** category → select **Price: Low to High** → collect displayed prices → verify ascending order	Product prices are sorted in ascending numerical order	`CollectionUtils.isAscending()` returned **false**, indicating incorrect sorting	Failed
verifyPriceHighToLowSorting	TC3	Select **Price: High to Low** → collect displayed prices → verify descending order	Product prices are sorted in descending numerical order	`CollectionUtils.isDescending()` returned **false**, indicating incorrect sorting	Failed
shouldAddThreeDistinctProductsAndVerifyCartTotals	TC4	Add three distinct products → verify cart badge → open cart → verify item count, subtotal, and order total	Cart badge displays **3** items; cart contains three line items; subtotal and total calculations are correct	As expected	Passed
shouldUpdateFirstLineItemQuantityAndVerifyCartTotals	TC5	Add three products → update the first item's quantity to **2** → verify updated totals → restore quantity to **1** → verify totals again	Line-item subtotal and order total update correctly after increasing and decreasing quantity	As expected	Passed

testEmptyCartByDeletingItemsFromCartPage	TC6	Add products → remove the first cart item repeatedly → verify row count decreases after each removal → verify empty cart message	Cart row count decreases by one after each deletion; empty cart message is displayed after the final item is removed	As expected	Passed